

C++ and Objected Oritend Programming

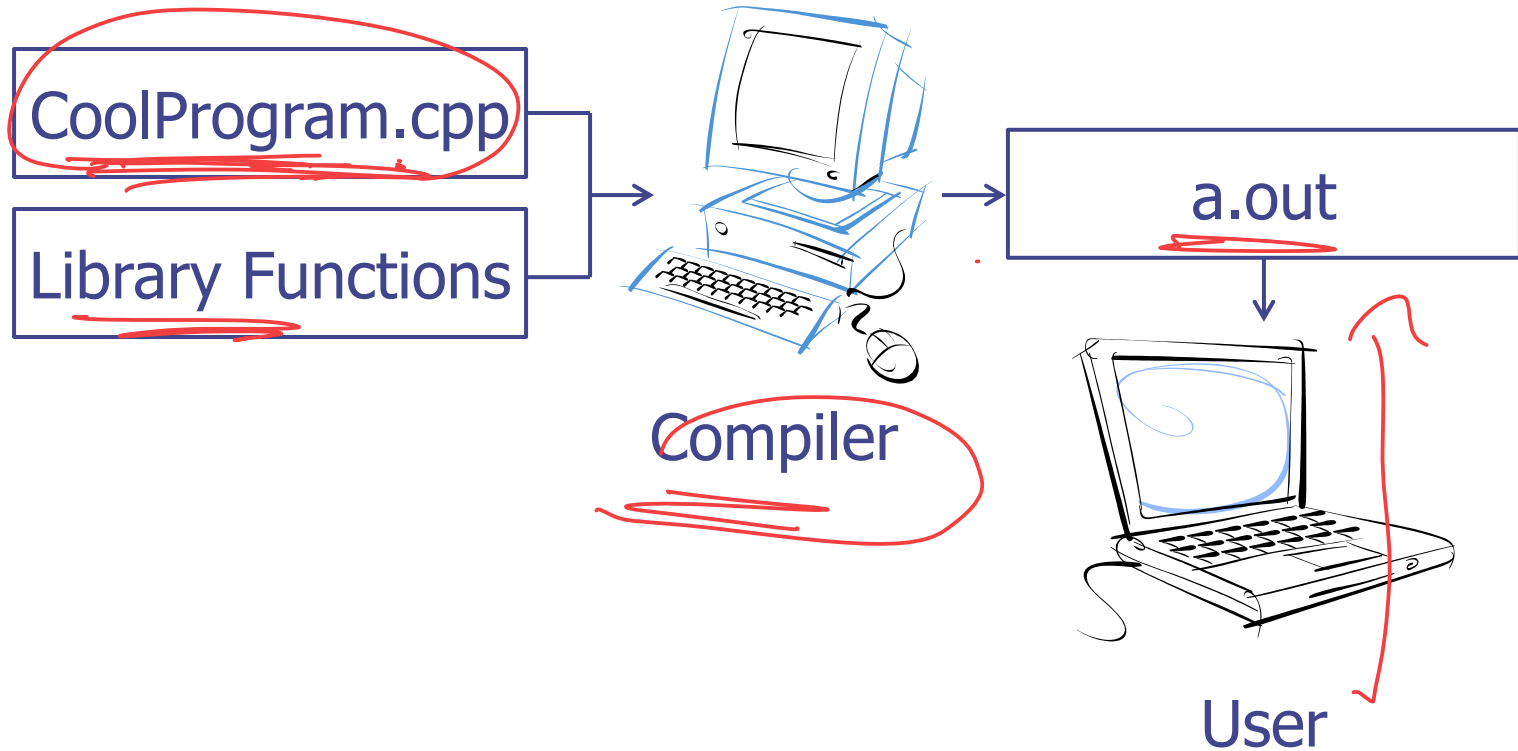
Manjanna B

Goals of This Lecture

- Overview of C++ language
 - ✓ At a glance, C++ = C + Class
- Intro to object-oriented (OO) programming
 - procedural or functional*
 - ✓ In structured programming, program = a series of functions
 - ✓ In OO programming, program = interaction between objects
 - ✓ OO encourages abstraction
 - ◆ Effective in representing a complex problem
 - ✓ OO encourages software reuse
 - ◆ Easily reuse classes and their implementation

Objected Oriented Programming

The C++ Programming Model



A Simple C++ Program

- Two integer inputs x and y
- Output their sum

```
#include <cstdlib>           stdlib.h  
#include <iostream>  
/* This program inputs two numbers x and y and outputs their sum */  
int main( ) {  
    int x, y;  
    std::cout << "please enter two numbers: "  
    std::cin >> x >> y;      // input x and y  
    int sum = x + y;          // compute their sum  
    std::cout << "Their sum is " << sum << std::endl;    " /n "  
    return EXIT_SUCCESS      // terminate successfully  
}
```

Abstraction and Abstract Data Type

- Abstraction: depends on what to focus
 - ✓ Procedure abstraction: focuses on operations
 - ✓ Data abstraction: data + operations as one
 - ✓ Object abstraction: data abstraction + reusable sub types (class)
- Abstract data type (ADT)
 - ✓ Definition of a set of data + associated operations
- Implementation of ADT
 - ✓ Data → data structure
 - ♦ Stack, Queue, Tree etc.
 - ✓ Operations → manipulation of data structure
 - ♦ Stack: push, pop etc.
 - ✓ Error conditions associated with operations

array linked list

Example of ADT

- Example: ADT modeling a simple stock trading system

- ✓ The data stored are buy/sell orders

- ✓ The operations supported are

- ♦ order buy(stock, shares, price)

- ♦ order sell(stock, shares, price)

- ♦ void cancel(order)

← in match ()

- ✓ Error conditions:

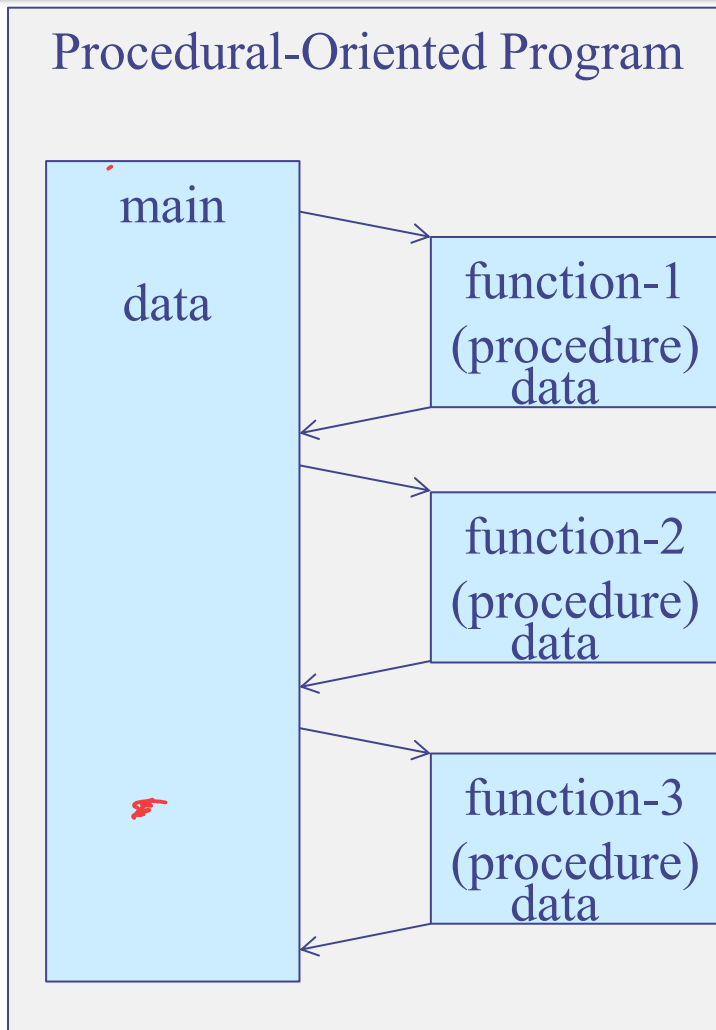
- ♦ Buy/sell a nonexistent stock

- ♦ Cancel a nonexistent order

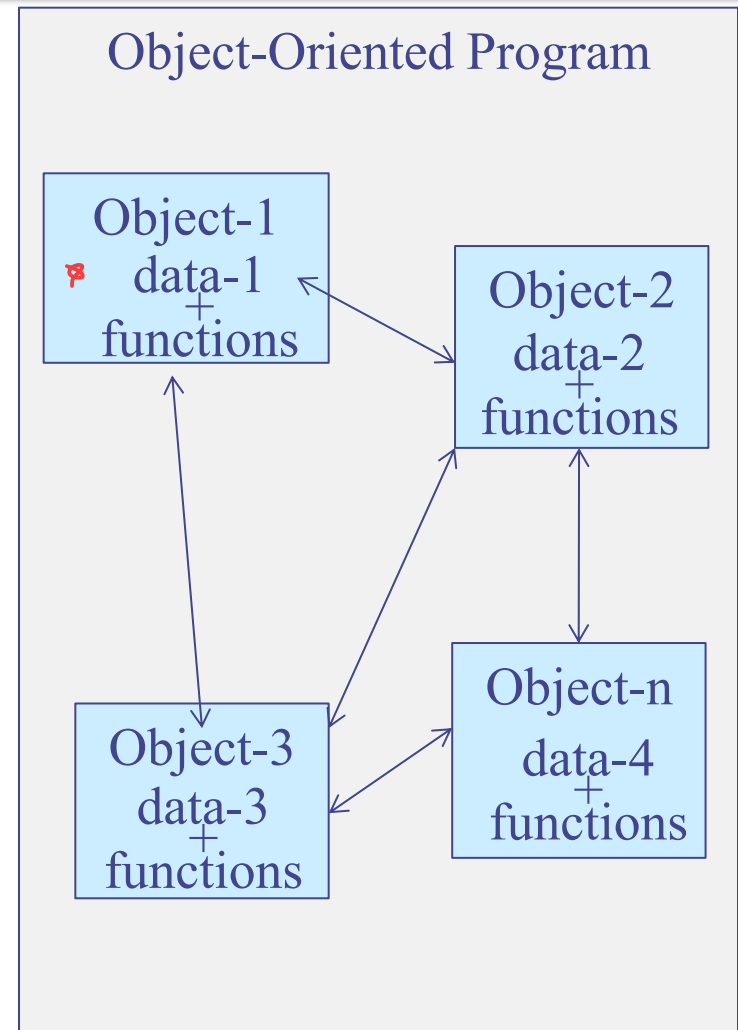
C & C++ in Abstraction View

- C supports Procedure-Oriented programming
 - ✓ Procedure (function) + data structure
 - ◆ Procedure (function) : manipulate data
- C++ supports Object-Oriented programming
 - ✓ Object-oriented programming (OOP) is a programming paradigm that uses objects and their interactions to design applications and computer programs.
 - ✓ Data abstract + reusable subtypes with following features
 - ◆ Encapsulation, Polymorphism, Inheritance

Procedural-Oriented VS. Object-Oriented



data is open to all functions.

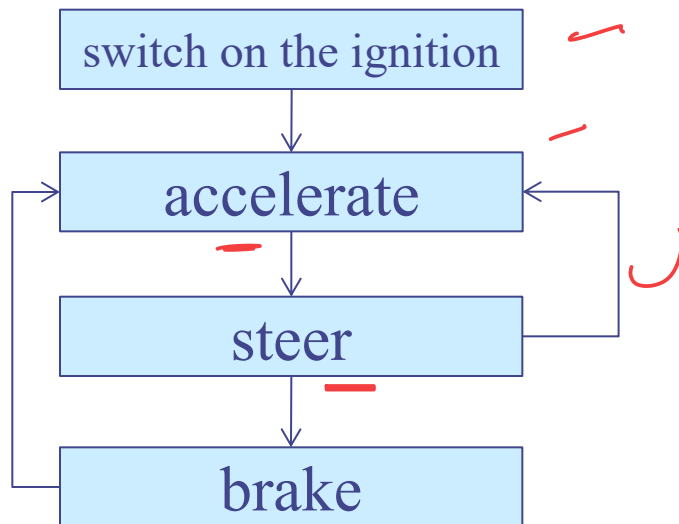


Each data is hidden and associated with an object.

Example: PO VS. OO

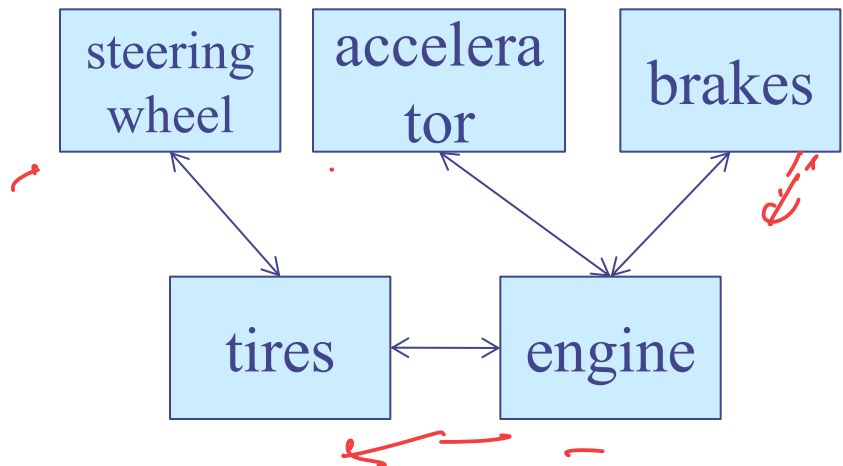


Procedure-oriented View of car operation



Car = a sequence of functions (procedures)

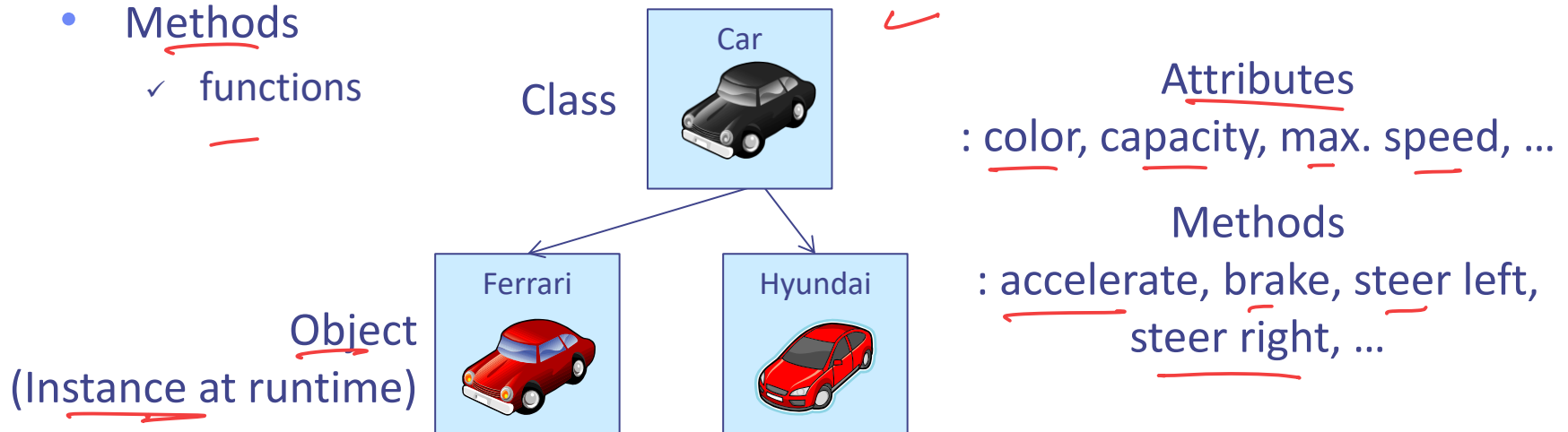
Object-oriented View of car operation



Car = interaction between components (objects)

What is Object ?

- Class ($\leftrightarrow$ Type in C)
 - ✓ Defines the abstract characteristics of a thing (object)
 - ♦ attributes (data) + behaviors (operations = methods)
- Object ($\leftrightarrow$ Variable in C)
 - ✓ A pattern (exemplar) of a class
- Instance
 - ✓ The actual object created at runtime
 - ✓ State: the set of values of the attributes of a particular object
- Methods
 - ✓ functions



C++ Classes

- Similar to structure in C
Class in C++

```
class class_name {  
  public:  
    // member variables  
    int a, b, c;  
    ...  
    // member methods (functions)  
    void print(void);  
    ...  
};
```

a collection of types and
associated functions

Structure in C

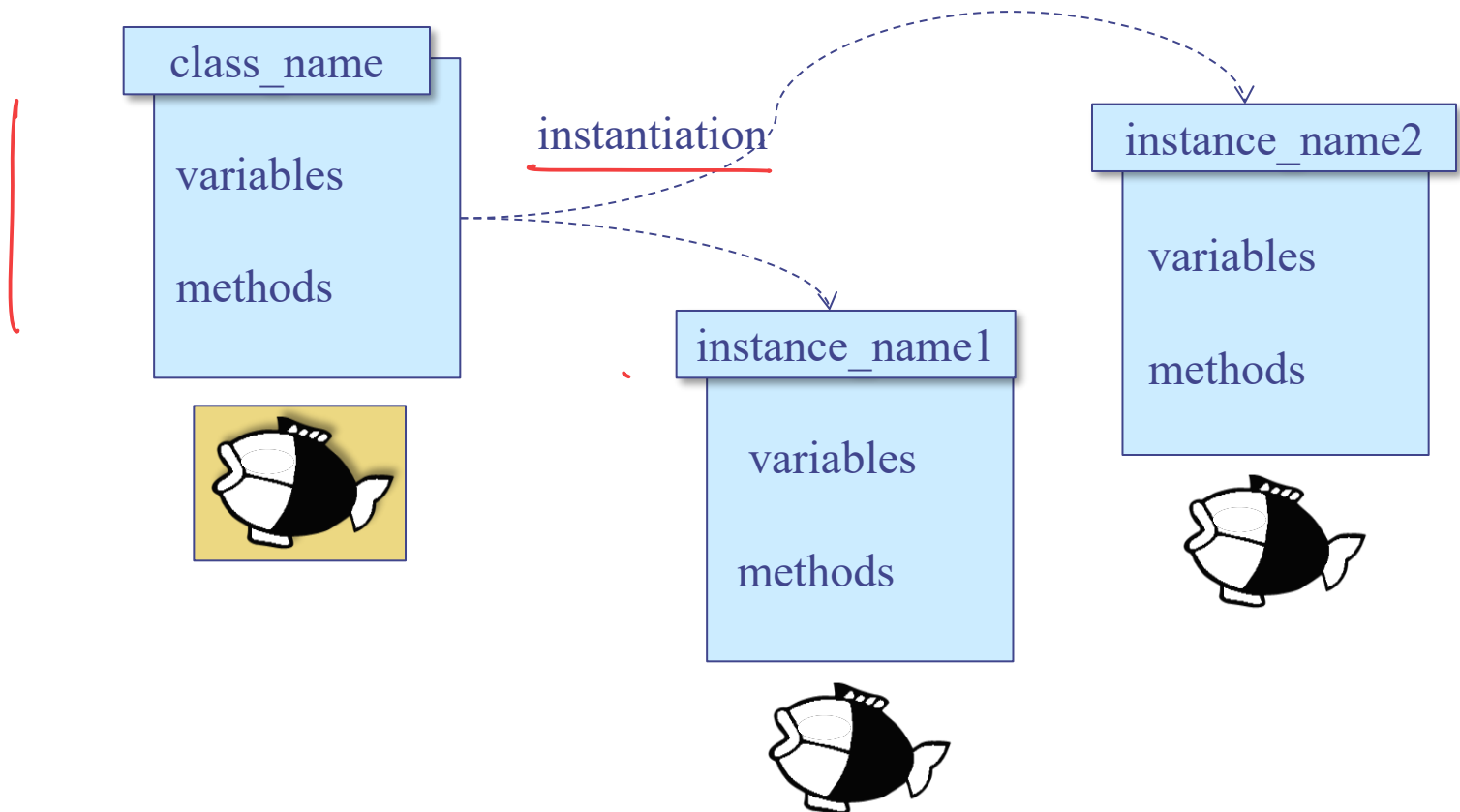
```
struct tag_name {  
    type1 member1;  
    type2 member2;  
    ...  
    typeN memberN;  
};
```

a collection of heterogeneous types

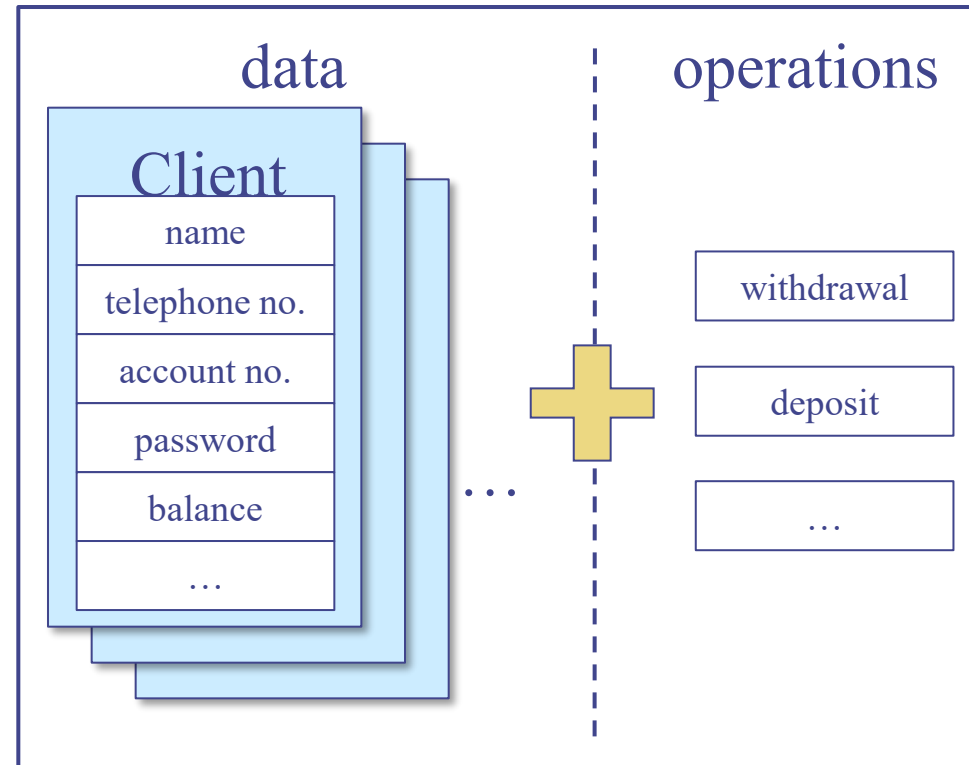
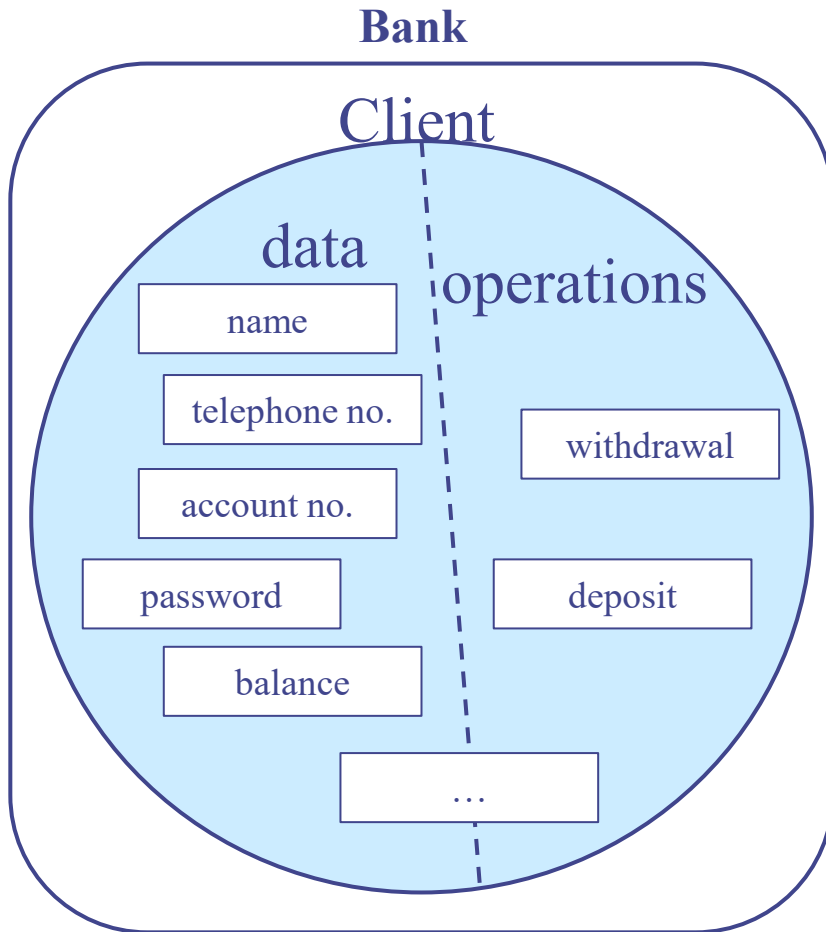
Class Declaration

class_name instance_name1, instance_name2;

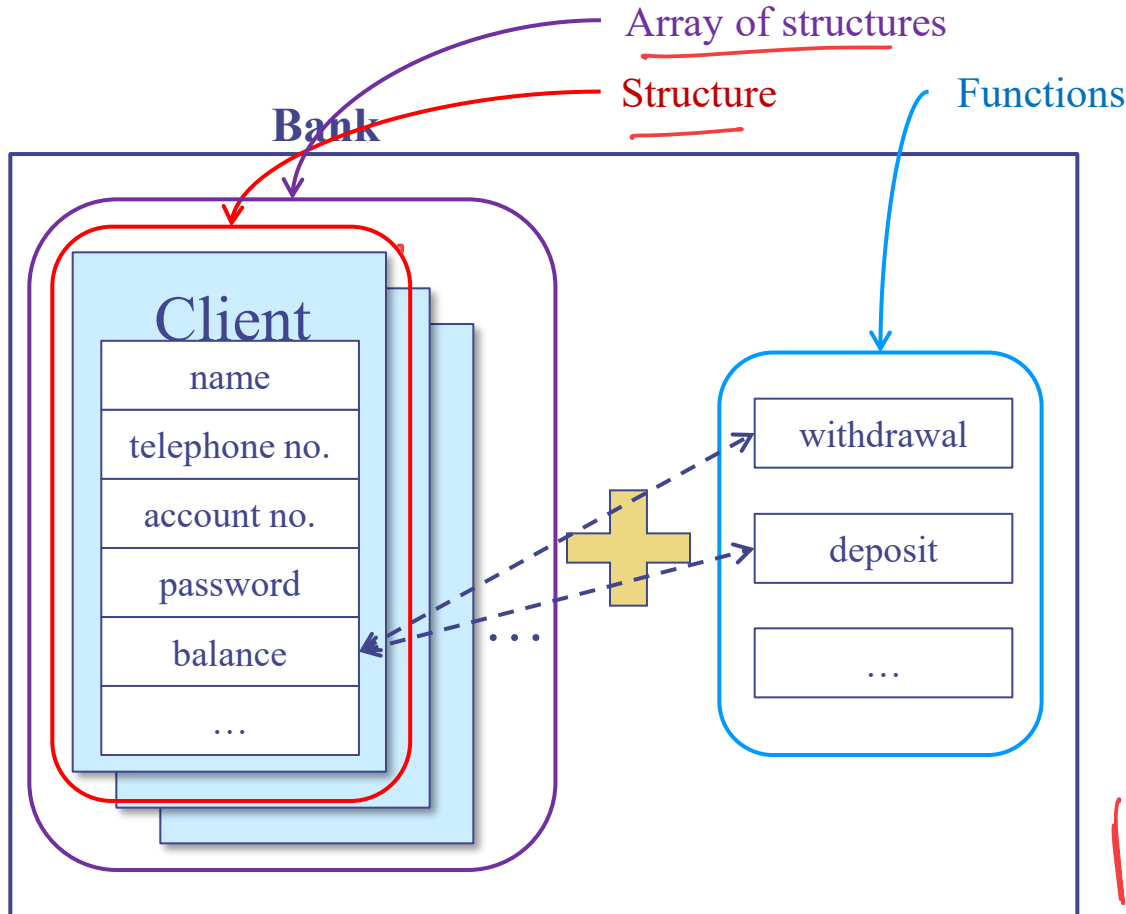
C.f. struct *tag_name* struct_variable, ... ;



C Style Design (Procedural) (1/2)



C Style Design (Procedural) (2/2)



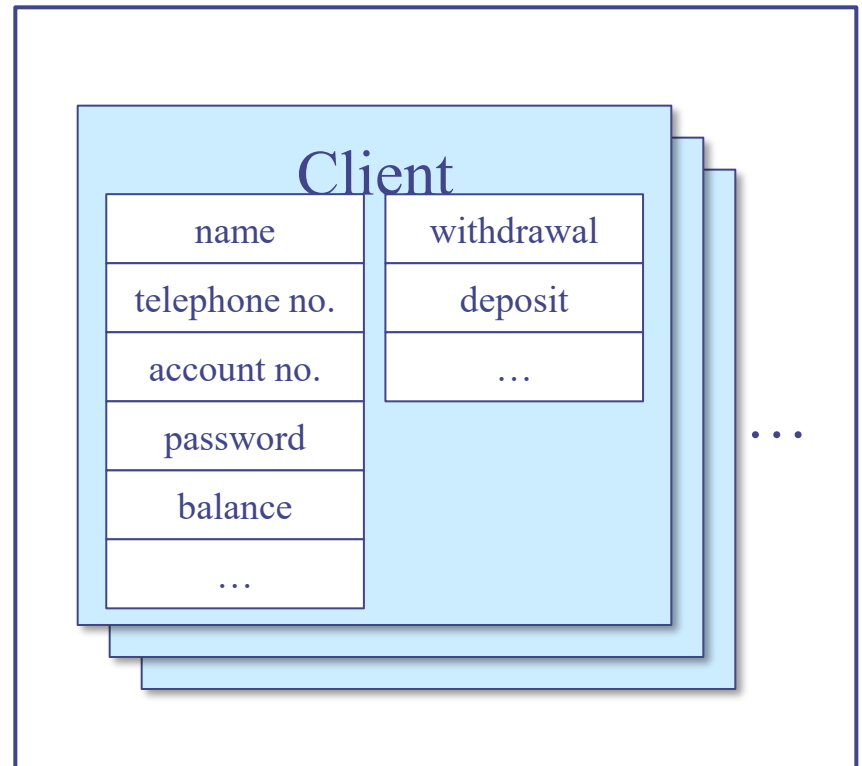
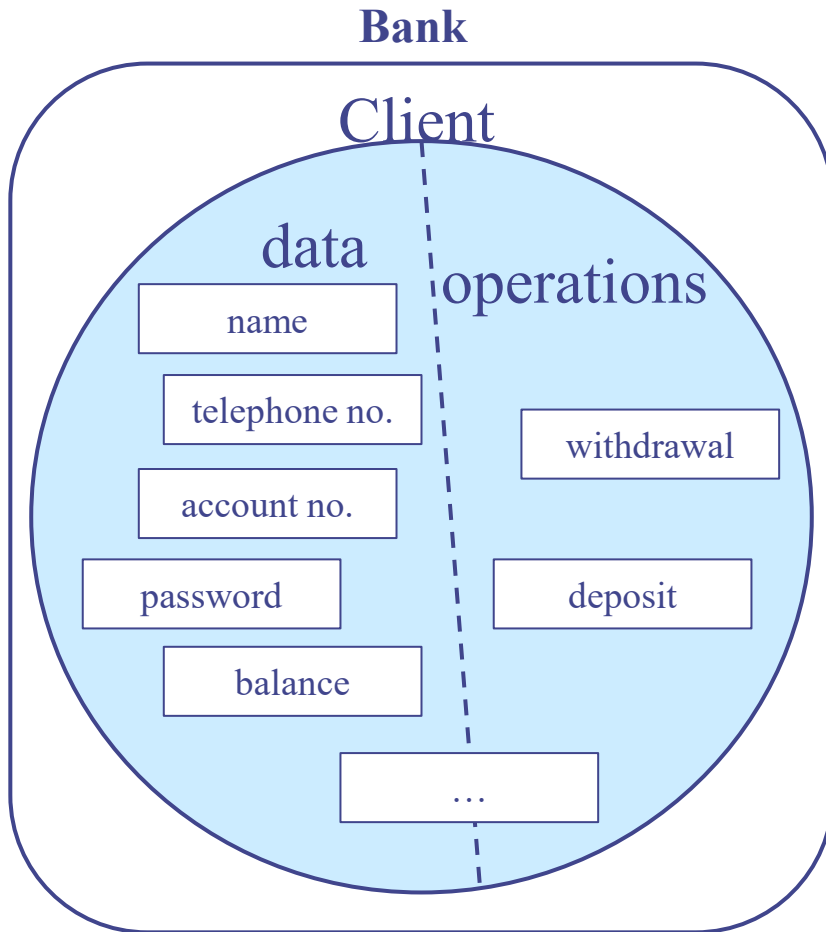
```
struct client {  
    char name[MAX];  
    char tel[MAX];  
    char account[MAX];  
    char password[MAX];  
    int balance;  
};
```

```
struct client clients[MAX_NO];
```

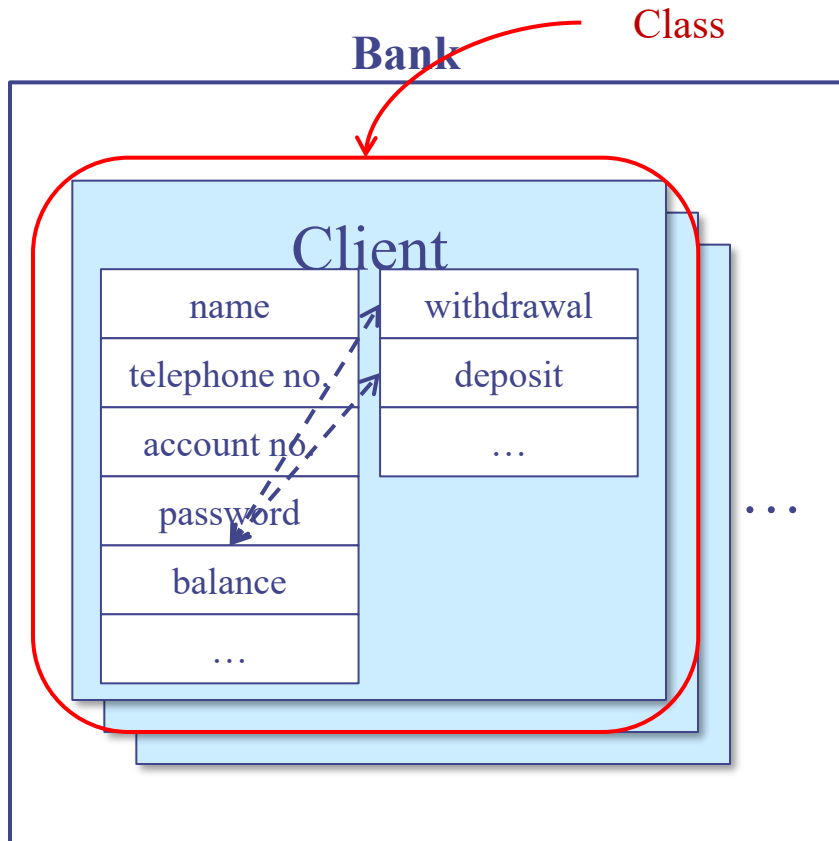
```
void withdrawal (client &cli, int money);  
void deposit (client &cli, int money);
```

references

C++ Style Design (Object-Oriented) (1/2)



C++ Style Design (Object-Oriented) (2/2)



```
class client {  
    char name[MAX];  
    char tel[MAX];  
    char account[MAX];  
    char password[MAX];  
    int balance;  
    void withdrawal (int money);  
    void deposit (int money);  
};
```

member variables
are not required

```
client clients[MAX_NO];
```

"struct" can be omitted in C++

In C++, structure is a **class with all members public**.

struct s { , , , } ≡ class s {public: , , , }

Example: Class

```
#include<iostream>
#define MAX 10
using namespace std;
```

```
class record{
```

```
public:
```

```
char name[MAX];
int course1, course2;
double avg;
```

```
void print(void) {
    cout << name << endl;
    cout << "course1 = " << course1
        << ", course2 = " << course2 << endl;
    cout << "avg = " << avg << endl;
}
```

```
};
```

instantiation

referencing
public member
variables

member variables

Access specifier

```
int main( ) {
```

```
record myrecord;
```

```
myrecord.name = "KIM JH";
```

```
myrecord.course1 = 100;
```

```
myrecord.course2 = 90;
```

```
int sum = myrecord.course1 +
          myrecord.course2;
```

```
myrecord.avg = ((double) sum) / 2;
```

```
myrecord.print( );
```

```
return 0;
```

```
}
```

member function call

member function

result>

KIM JH

course1 = 100, course2 = 90

avg = 95

Definition of Member Functions

whole code in same file
ex) "record.cpp"

```
class record{  
public:  
    char name[MAX];  
    int course1, course2;  
    double avg;  
    void print(void) {  
        cout << name << endl;  
        cout << "course1 = " << course1  
            << ", course2 = " << course2  
        << endl;  
        cout << "avg = " << avg << endl;  
    }  
};
```

declaration & definition

```
class record{  
public:  
    char name[MAX];  
    int course1, course2;  
    double avg;  
    void print(void);  
};
```

declaration
definition "record.h"
always after declaration

```
void record::print(void) {  
    cout << name << endl;  
    cout << "course1 = " << course1  
        << ", course2 = " << course2 << endl;  
    cout << "avg = " << avg << endl;  
}
```

"record.cpp"

- don't miss #include "record.h" in "record.cpp"

Member Variables & Functions

```
#include<iostream>
#define MAX 10
using namespace std;
```

```
class record{
public:
```

```
    char name[MAX];
    int course1, course2;
    double avg;
```

```
    void print(void) {
        cout << name << endl;
        cout << "course1 = " << course1
            << ", course2 = " << course2
            << endl;
        cout << "avg = " << avg << endl;
    }
};
```

always must reference
member variables
with instance name

```
int main( ) {
```

```
    record myrecord;
```

```
    myrecord.name = "KIM JH";
```

```
    myrecord.course1 = 100;
```

```
    myrecord.course2 = 90;
```

```
    int sum = myrecord.course1 +
              myrecord.course2;
```

```
    myrecord.avg = ((double) sum) / 2;
```

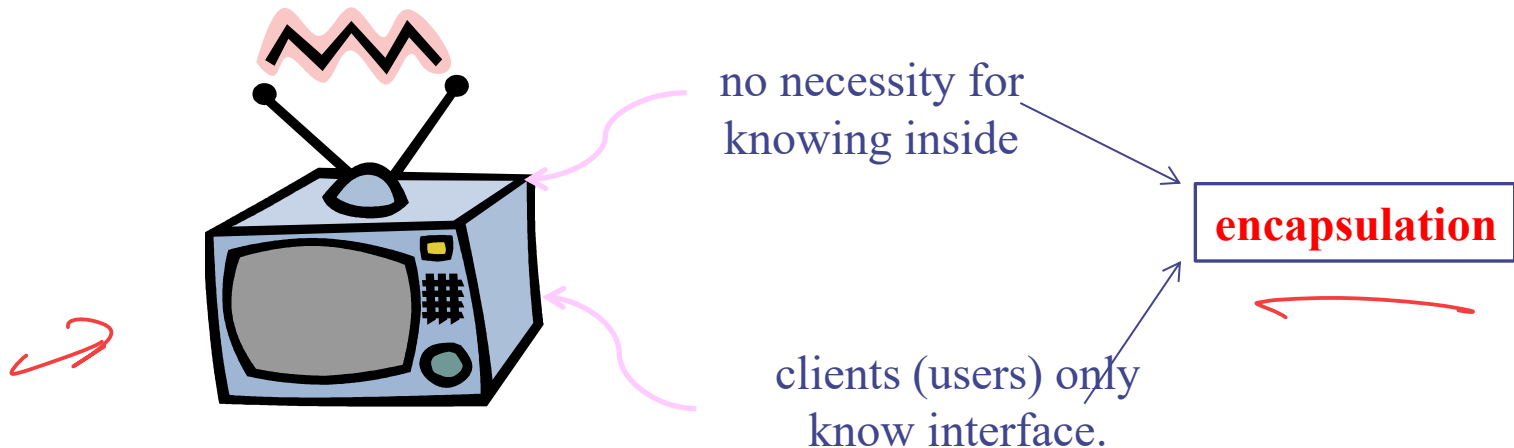
```
    myrecord.print( );
```

```
    return 0;
```

can reference member variables
without class name
inside member functions

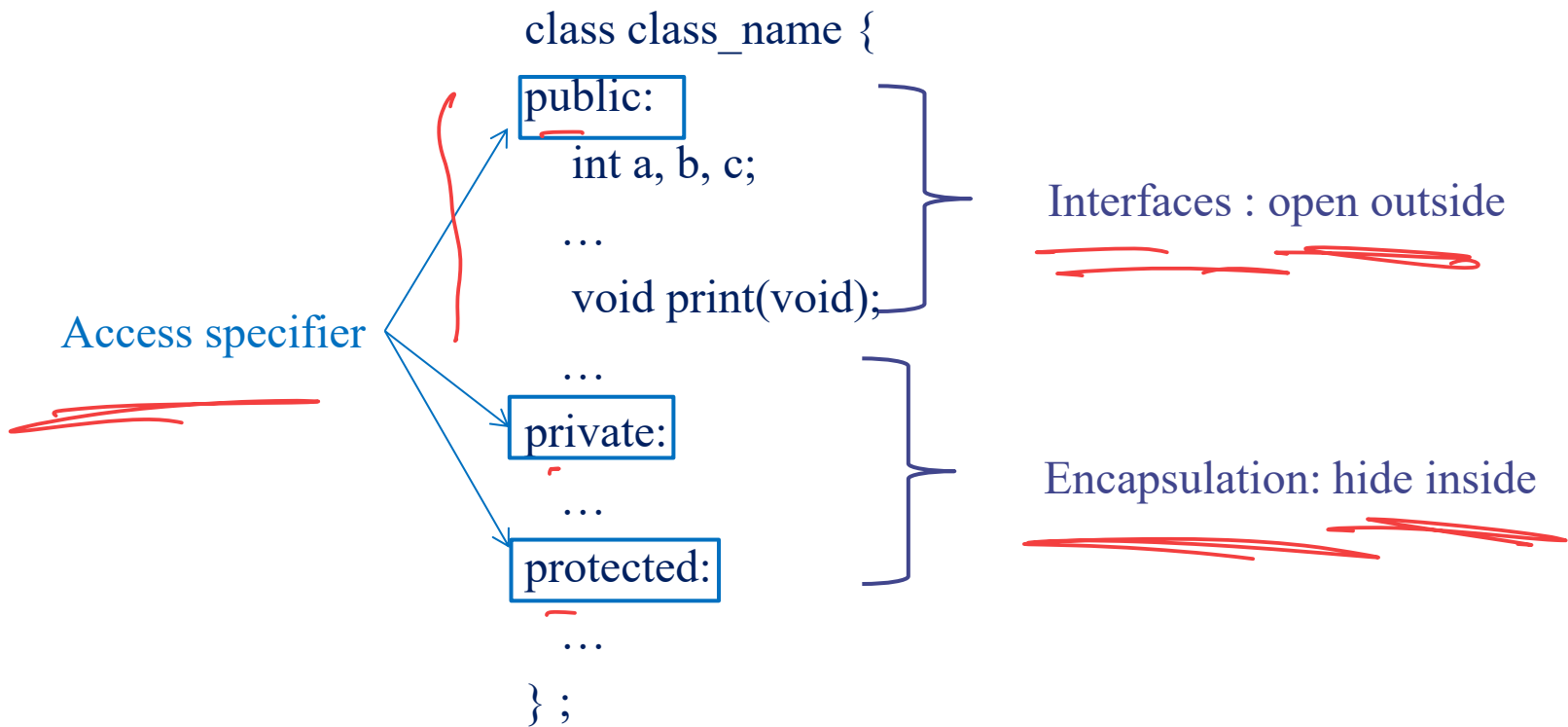
Encapsulation

- Encapsulation conceals the functional details defined in a class from external world (clients).
 - ✓ Information hiding
 - ◆ By limiting access to member variables/functions from outside
 - ✓ Operation through interface
 - ◆ Allows access to member variables through interface
 - ✓ Separation of **interface from implementation**
 - ◆ Similar to Stack data type and implementation (Lecture 11)



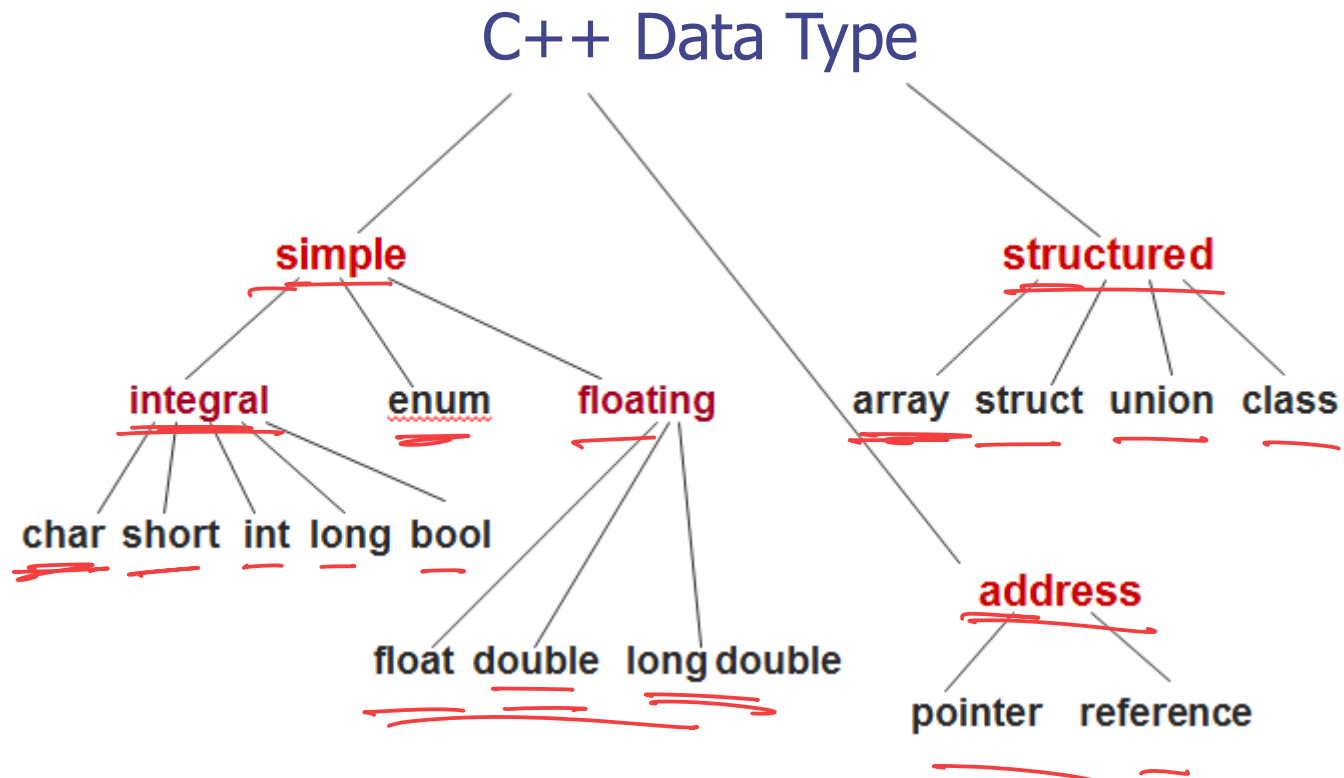
Encapsulation in C++

Class in C++



Basic Features (Mostly same as C)

C++ Data Types



Fundamental Types

- Basic data types

- ✓ bool Boolean value, either true or false
- ✓ char Character "\0"
- ✓ short Short integer
- ✓ int Integer
- ✓ long Long integer — F
- ✓ float Single-precision floating-point number
- ✓ double Double-precision floating-point number
- ✓ enum User-defined type, a set of discrete values
- ✓ void The absence of any type information

Declaration of a Variable

- We can provide a definition, or initial value
- Without definition, initial value is zero
- Variable names may consist of any combination of letters, digits, or the underscore (_) character, but the first character cannot be digit
- ex)

```
short n;  
int octalNumber = 0400;  
char newline_character = '\n';  
long BIGnumber = 314159265L;  
short _aSTRANGE__1234_variABIE_NaMe;
```

Characters: char

- Typically 8-bit
- Literal
 - ✓ A constant value appearing in a program
 - ✓ Enclosed in single quotes
 - ✓ A backslash (\) is used to specify a number of special character literals

<code>'\n'</code>	newline	<code>'\t'</code>	tab
<code>'\b'</code>	backspace	<code>'\r'</code>	return
<code>'\0'</code>	null	<code>'\"'</code>	single quote
<code>'\"'</code>	double quote	<code>'\\'</code>	backslash

Integers: short, int, long

- Short int, (plain) int, long int
- Decimal numbers
 - ✓ ex) 0, 25, 98765, -3
- Suffix “l” or “L” indicate a long integer
 - ✓ ex) 123456789L
- Prefix “0” indicates octal constants
 - ✓ ex) 0400 (256) $0400 = 4 \times 8^2 = 256$
- Prefix “0x” indicates hexadecimal constants
 - ✓ ex) 0x1c (28)
 $1 \times 16^1 + 12 \times 16^0$

Floating Point: float, double

- Floating point literals
 - ✓ ex) 3.14159, -1234.567, 3.14E5, 1.28e-3
- Default is double type
- Suffix “f” or “F” indicate float
 - ✓ ex) 2.0f, 1.234e-3F

1.28 × 10⁻³

1.234 × 10⁻³

Enumerations: enum

- A user-defined type that can hold any of a set of discrete values
- Once defined, enumerations behave much like an integer type
- Each element of an enumeration is associated with an integer value
- ex)

```
enum Color {RED, GREEN, BLUE}; //RED=0, GREEN=1, BLUE=2
enum Mood {HAPPY=3, SAD=1, ANXIOUS=4, SLEEPY=2};
```

```
Color skycolor = BLUE;
Mood myMood = SLEEPY;
```

Pointers

- Pointer holds the value of an memory address
 - The type T^* denotes a pointer to a variable of type T
 - ✓ ex) int^* , char^*
 - The 'address-of' operator, '&', returns the address of a variable
 - Dereferencing
 - ✓ Accessing the object addressed by a pointer
 - ✓ Done by $*$ operator
-
- Handwritten notes and diagram illustrating pointer operations:
- Code snippets: $\text{int}^* \text{ptr};$, $\text{int } n;$, $\text{ptr} = \&n;$
 - Memory layout diagram:
 - Addresses: 1001, 1002, 1003, 1004
 - At address 1001, a box contains the value 2000. A red arrow labeled n points to this box.
 - Below, a box labeled ptr contains the value 1001. A red arrow points from this box to the box at address 1001.
 - Below the ptr box, the text $\text{ptr} = \text{ptr} + 1;$ is written. A red arrow points from this text to the value 1005 below it.
 - At the bottom, the text $* \text{ptr}$ is written. A red arrow points from the ptr box to this text.

Pointers

- ex)

```
char ch = 'Q';  
char* p = &ch;    // p holds the address of ch  
cout << *p;       // outputs the character 'Q'  
ch = 'Z';          // ch now holds 'Z'  
cout << *p;       // outputs the character 'Z'
```

- Null pointer points to nothing
- Void type pointer can point to a variable of any type
- Cannot declare a void type variable

```
void * ptr;  
int n = 50;  
ptr = &n;  
int * ptr1 = (int *) ptr;
```

```
void * ptr;  
int n = 50;  
ptr = &n;  
int m = (int) (*ptr);
```


- A void pointer (void*) is a generic pointer that can store the address of any type of variable.

```
#include <iostream>
using namespace std;

int main() {
    int x=10;
    float y= 3.14f;
    char ch = 'A';
    void *ptr;
    ptr = &x;
    cout << *(int*) ptr << endl;
```

```
ptr = &y;
cout << *(float*) ptr << endl;
```

```
ptr = &ch;
cout << *(char*) ptr << endl;
return 0;
}
```

- void *ptr can point to an int, float, or char.
- Since a void* has no type information, it must be typecast before dereferencing.

Arrays

- A collection of elements of the same type
- Index references an element of the array
- Index is a number from 0 to N-1
- ex)



```
double f[3];           // array of 3 doubles: f[0], f[1], f[2]
double* p[10];         // array of 10 double pointers: p[0], ... , p[9]
f[2] = 25.3;
p[4] = &f[2];           // p[4] points to f[2]
cout << *p[4];          // outputs "25.3"
```

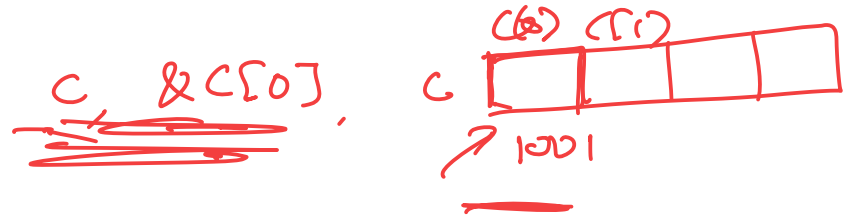
Arrays

- Two-dimensional array
 - ✓ An “array of arrays”
 - ✓ ex) int A[15][30]
- Initializing
 - ✓ ex)

```
int a[4] = {10, 11, 12, 13}; // declares and initializes a[4]
bool b[2] = {false, true}; // declares and initialize b[2]
char c[] = {'c', 'a', 't'}; // declares and initialize c[3]
// compiler figures the size of c[]
```

Pointers and Arrays

- The name of an array can be used as a pointer to the array's initial element and vice versa
- ex)



```
✓ 0 1 2
char c[] = {'c', 'a', 't'};
char *p = c;           // p point to c[0]
char *q = &c[0];       // q also points to c[0]
cout << c[2] << p[2] << q[2] // outputs "ttt"
```

C-Style Structure

- Storing an aggregation of elements which can have different types
- These elements called “member” or “field”, is referred to by a given name
- ex)

```
enum MealType { NO_PREF, REGULAR, LOW_FAT, VEGETARIAN };
```

```
struct Passenger {
```

```
    string    name;           // possible value: "John Smith"
```

```
    MealType  mealPref;       // possible value: VEGETARIAN
```

```
    bool      isFreqFlyer;     // possible value: true
```

```
    string    freqFlyerNo;     // possible value: "293145"
```

```
};
```

C-Style Structure

- This defines a new type called Passenger
- Declaration and initialization

✓ ex)

```
Passanger pass = { "John Smith", VEGETARIAN, true, "293145" }
```

- Member selection operator

✓ struct_name.member

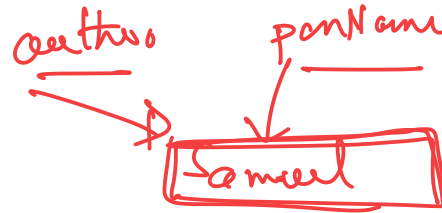
✓ ex)

```
pass.name = "Pocahontas";           // change name  
pass.mealPref = REGULAR;           // change meal preference
```

- This is just for backward-compatibility
- "Class" is much more powerful

References

- An alternative name for an object (i.e., alias)
- The type T& denotes a reference to an object of type T
- Cannot be NULL
- ex)



```
string author = "Samuel Clemens";  
string &penName = author;           // penName is an alias for author  
penName = "Mark Twain";             // now author = "Mark Twain"  
cout << author;                     // outputs "Mark Twain"
```

Constants

- Adding the keyword const to a declaration
- The value of the associated object cannot be changed

- ex)

```
const double PI = 3.14159265;  
const int CUT_OFF[] = {90, 80, 70, 60};  
const int N_DAYS = 7;  
const int N_HOURS = 24*N_DAYS;           // using a constant expression  
int counter[N_HOURS];                   // constant used for array size
```

- Replace "#define" in C for the definition of constants

#define max 100

Typedef

- Define a new type name with keyword typedef
- ex)

```
typedef char* BufferPtr;           // type BufferPtr is a pointer to char
typedef double Coordinate;      // type Coordinate is a double

BufferPtr p;                      // p is a pointer to char
Coordinate x, y;                 // x and y are of type double
```

Dynamic Memory Allocation

Dynamic Memory and 'new' Operator

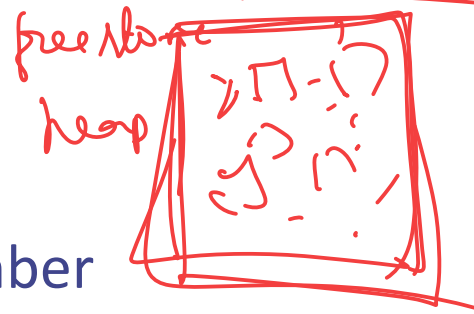
- Create objects dynamically in the 'free store'
- The operator 'new' dynamically allocates the memory from the free store and returns a pointer to this object

- Accessing members

- ✓ pointer_name->member

- ✓ (*pointer_name).member

- ✓ Same as how to access a member in C Structure



malloc()
calloc()
realloc()
free()

- The operator 'delete' operator destroys the object and returns its space to the free store

Dynamic Memory and 'new' Operator

- ex)

```
Passenger *p;
```

```
//...  
```

```
p = new Passenger;
```

```
p->name = "Pocahontas";
```

```
p->mealPref = REGULAR;
```

```
p->isFreqFlyer = false;
```

```
p->freqFlyerNo = "NONE";
```

```
//...
```

```
delete p;
```



```
// p points to the new Passenger
```

```
// set the structure members
```

```
// destroy the object p points to
```

Example: Operators for Dynamic Allocation

C

Functions

```
void * malloc ( size_t size )  
void * calloc (size_t nmemb, size_t size )  
void free(void *ptr);
```

Ex) To allocate a char

C

```
char *cptr;  
cptr = (char *) malloc(sizeof(char));  
...  
free(cptr);
```

Ex) To allocate an integer array of 100 elements

C

```
int *iptr;  
iptr = (int *) calloc(100, sizeof(int));  
...  
free(iptr);
```

C++

Operators

```
new data_type  
new data_type[size] ←  
delete scalar_variable;  
delete [] array_variable;
```

returns a pointer
addressing the 1st
element of the array

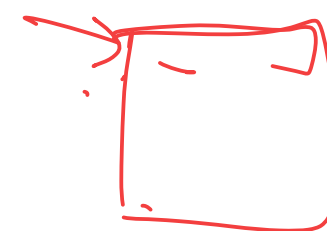
C++

```
char *cptr = new char;  
...  
delete cptr;
```

C++

```
int *iptr = new int[100];  
...  
delete [] iptr;
```

size of(int) x 100



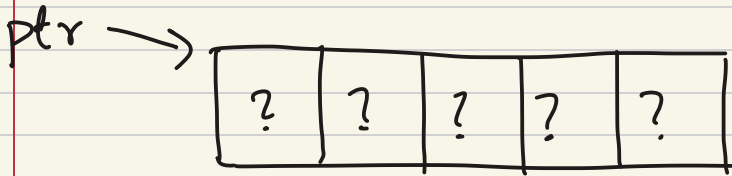
Questions

- How to dynamically allocate “array of pointers”?
- How to declare two dimensional matrix (i.e., matrix) and dynamically allocate its space?
- You can use your own method, but you can also use ‘vector’ class in STL library

How to dynamically allocate "array of pointers"?

`int *ptr[5];` // static array of pointers

`int **ptr = new int*[5];` // dynamic allocation
// pointer to a dynamically allocated array



each element is an `int*`.

- How to declare two dimensional matrix (i.e., matrix) and dynamically allocate its space?

```
int rows = 3;
```

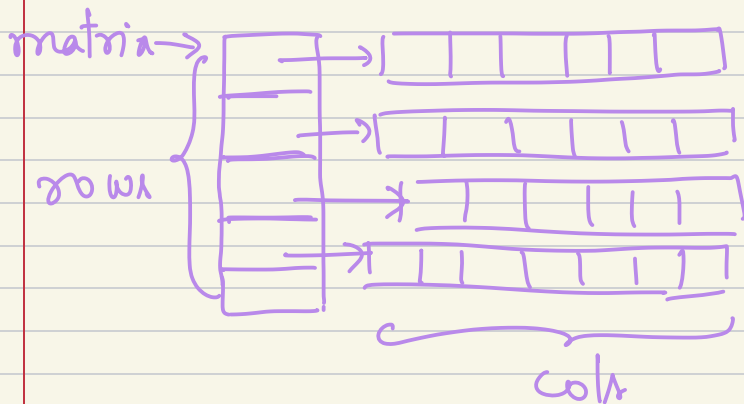
```
int cols = 4;
```

```
int **matrix = new int*[rows];
```

// allocate row pointers

```
for (int i=0; i<rows; i++)
```

```
    matrix[i] = new int[cols];
```



```
matrix[0][0] = 20;
```

```
matrix[1][2] = 25;
```

```
matrix[2][3] = 50;
```

```
for (int i=0; i<rows; i++)
```

```
    delete[] matrix[i];
```

```
delete[] matrix;
```


- You can use your own method, but you can also use 'vector' class in STL library

— Modern C++ prefers vector because it automatically manages memory.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    int rows=3;
    int cols=4;
    vector<vector<int>> matrix(rows, vector<int>(cols));
    for (int i=0; i<rows; i++)
        for (int j=0; j<cols; j++)
            matrix[i][j] = i+j;

    return 0;
}
```

vector is the class template provided by the STL
vector<int> is an instantiation of that template
vector<vector<int>> is another instantiation,
where each element is a vector<int>

Memory Leaks

- C++ does not provide automatic garbage collection
- If an object is allocated with new, it should eventually be deallocated with delete
- Deallocation failure can cause inaccessible objects in dynamic memory, memory leak